

# 1주차 동아리

## 파이썬 기초

### 변수 선언

- C언어에서는 변수는 반드시 데이터 타입과 길이를 명시해야한다.
- Python에서는 변수를 선언할 때 데이터 타입이나 길이를 명시할 필요가 없으며, 이를 동적 타이핑이라고 한다.

#### C언어 변수 선언 방식

```
int a =3;
float b = 3.14;
char c[100] = "안녕";
```

C

#### 파이썬 변수 선언 방식

```
a = 3
b = 3.14
c = '안녕'
```

PYTHON

### python 문법

- C언어에서는 코드 한 줄의 종료를 세미콜론(;)으로 표기하지만, Python에서는 줄바꿈(Enter)로 이를 표기한다.
- 또한 반복문&조건문의 블록을 작성할 때 C언어는 중괄호({})를 사용하여 표기하지만, Python에서는 들여쓰기(tab 또는 스페이스바)로 이를 표기한다.

#### C언어

C

```
int main(){
int i = 3;

if (i == 3){
    printf("i는 3입니다.\n");
    while (i > 7){
printf("i는 %d", i)}
return 0;
    }
}
```

## 파이썬

PYTHON

```
i = 3
if i == 3:
    print("i는 3입니다. \n")

    for i in range(5):
        print(f"i는 {i}")
```

## 자료 구조

### 기본 자료형

- 파이썬에서 단일 값을 갖는 자료형을 기본 자료형이라고 하며, 음수와 양수의 구분이 없다.
- C언어에서는 문자와 문자열이 다른 자료형이었지만, 파이썬에서는 모두 문자열 취급한다.
- C언어와 동일하게 = 기호를 활용하여 대입이 가능하다.

PYTHON

```
int_type = 10
float_type = 3.14
bool_Type = True
str_type = "안녕하세요"
None_Type = None
```

#### 자료형

#### 설명

|              |         |                                            |
|--------------|---------|--------------------------------------------|
| <b>int</b>   | 정수형     | 1, -1, 4 같은 정수를 표현하는데 사용                   |
| <b>float</b> | 실수형     | 3.14, -0.13같은 실수 표현                        |
| <b>bool</b>  | Boolean | 참/거짓을 표현하기 위한 자료형으로 True/False 값만 들어갈 수 있음 |

| 자료형                    | 설명                                                                           |
|------------------------|------------------------------------------------------------------------------|
| <code>str</code>       | 문자열 <code>"Hello", "Hi"</code> 와 같은 문자열, <code>"", ''</code> 으로 문자열을 선언 가능하다 |
| <code>none_Type</code> | 변수의 초기값이나 함수에서 반환할 값이 없는 경우 사용되는 특별 상수                                       |

## 복합 자료형

- C언어에서의 배열처럼 Python에서도 여러 값을 저장할 수 있는 자료형이 있다. 이를 복합 자료형이라고 한다.
- list, tuple, dict 등이 있다.
- list는 `[]` 기호를 사용하여 선언할 수 있으며, 빈 리스트를 생성하는 것도 가능하다. C언어와 달리 서로 다른 타입의 데이터들을 하나의 리스트에 저장하는 것도 가능하다.

PYTHON

```
list_type1 = []
list_type2 = [1, 2, 3]
list_type3 = [1, 3.14, 'a']
```

## 변수 선언

- 기본적으로 변수명은 대소문자를 구분하며, `문자, /숫자/( _ )` 기호를 포함할 수 있지만 숫자로 시작할 수 없다.
- True, False, or, break 등의 예약어는 변수명으로 사용이 불가능하다.

PYTHON

```
myVar = 10
MyVar = 10 # 이 둘은 다른 변수

Var2 = 11
_var = 11
var_2 = 11

True ; False; or; break; # 예약어는 변수로 사용 x
```

## 파이썬 입출력

### input 함수

- input은 사용자가 키보드로 입력한 모든 것을 문자열로 저장한다.

```
a = input()
>>>Life is too short, you need python
a
>>>Life is too short, you need python
```

- 프롬프트를 띄워 사용자 입력 받기  
사용자에게 입력 받을 때 '숫자를 입력하세요'나 '이름을 입력하세요'처럼 안내 문구를 보여 주고 싶을 때가 있을때 `input()`의 괄호 안에 안내 문구를 입력하면 된다.

```
number = input("숫자를 입력하세요: ")
```

- 괄호 안에 입력한 문구가 프롬프트로 나타난다.
- 여기서 주의 할 부분은 `input`은 입력되는 모든 것을 문자열로 취급한다. 따라서 `number`는 숫자가 아니라 문자열이다.

## 입력 값을 숫자로 변환

- `input` 함수는 항상 문자열을 반환하므로, 숫자 계산을 하려면 적절한 자료형으로 변환해야 한다.

```
#정수 변환
age = input("나이를 입력하세요")
age = int(age)
# 실수 변환
height = input("키를 입력하세요(cm): ")
float(height)
# 한번에 변환
age = int(input("나이를 입력하세요: "))
```

- 자료형 형태 변환

| 함수                   | 변환 결과 | 예시                                                         |
|----------------------|-------|------------------------------------------------------------|
| <code>int()</code>   | 정수    | <code>int("123") → 123</code>                              |
| <code>float()</code> | 실수    | <code>float("3.14") → 3.14</code>                          |
| <code>str()</code>   | 문자열   | <code>str(100) → "100"</code>                              |
| <code>bool()</code>  | 참/거짓  | <code>bool(1) → True</code> , <code>bool(0) → False</code> |
| <code>list()</code>  | 리스트   | <code>list("abc") → ['a', 'b', 'c']</code>                 |
| <code>tuple()</code> | 튜플    | <code>tuple([1, 2]) → (1, 2)</code>                        |
| <code>set()</code>   | 집합    | <code>set([1, 1, 2]) → {1, 2}</code>                       |
| <code>dict()</code>  | 딕셔너리  | <code>dict([("a", 1), ("b", 2)]) → {'a': 1, 'b': 2}</code> |

## print 함수

print함수의 용도는 데이터를 출력하는 것이다.

PYTHON

```
a = 123
print(a)
>>>123
a = "Python"
print(a)
>>>Python
a = [1, 2, 3]
print(a)
>>>[1, 2, 3]
```

따옴표로 둘러싸인 문자열은 + 연산과 동일하다

PYTHON

```
print("I" "Love" " YOU")
>>>ILove YOU
print("I"+"Love"+" YOU")
>>>ILove YOU

a = "I"
b = " Love"
c = " You"
print(a, b, c)
>I Love You
```

- `[", ']'`로 둘러싸인 문자열을 연속해서 쓰면 + 연산을 한 것과 같다.

여러 값들 출력하기

- print 함수 내에 콤마(,)를 넣어서 한 번에 여러 변수나 값을 출력할 수 있다. 콤마를 넣으면 각 값 사이에 공백 한 칸씩 띄워진 상태로 출력이 된다.

```
print(1, 2, 3)
>1 2 3

a = 3
b = 4
c = 5

print(a, b, c)
> 3, 4, 5
```

- 각 값 사이에 공백이 아니라 특정한 문자를 넣을 수 도 있다. 이 경우 `sep` 라는 구분자를 이용하나.

```
print(1, 2, 3, sep=',')
>1,2,3

print(1, 2, 3, sep='ab')
>1ab2ab3ab

print(1,2,3, sep='\n')
>1
>2
>3
```

- 줄 바꿈 생략하기: 파이썬 print 함수는 출력을 한 후 자동으로 줄 바꿈을 수행한다. 하지만 `end` 라는 옵션을 수행하면 출력의 끝에 오는 문자를 지정할 수 있다. 즉 줄 바꿈을 하지 않고 이어서 출력할 수 있다.

```
print(1,2,3, end= '')
print(4, 5, 6)
>1 2 3 4 5 6

print(1,2,3, end= ' and ')
print(4, 5, 6)
>1 2 3 and 4 5 6
```

예제

```
=== 간단한 계산기 ===
```

```
첫 번째 숫자를 입력하세요: 10
```

```
두 번째 숫자를 입력하세요: 3
```

```
10.0 + 3.0 = 13.0
```

```
10.0 - 3.0 = 7.0
```

```
10.0 * 3.0 = 30.0
```

```
10.0 / 3.0 = 3.3333333333333335
```

해당 결과가 나오게 하는 코드를 직접 짜보기

## 연산자

산술 연산자의 종류

PYTHON

```
result = 15
(result > 10) and (result < 200)
```

| 연산자 | 의미     | 사용예        | 설명                           |
|-----|--------|------------|------------------------------|
| =   | 대입 연산자 | a = 3      | 정수 3을 a에 대입                  |
| +   | 더하기    | a = 3 + 4  | 3과 4를 더한 값을 a에 대입            |
| -   | 빼기     | a = 3 - 4  | 3과 4를 뺀 값을 a에 대입             |
| /   | 곱하기    | a = 3 * 4  | 3과 4를 곱한 값을 a에 대입            |
| //  | 나누기(몫) | a = 3 // 4 | 3과 4를 나눈 후 소수점을 버리고 값을 a에 대입 |
| %   | 나머지값   | a = 3 % 4  | 3을 4로 나눈 후 나머지 값을 a에 대입      |
| **  | 제곱     | a = 3 ** 4 | 3의 4제곱을 a에 대입                |

## 관계 연산자

PYTHON

```
a, b = 10, 20
print(a == b, a != b, a > b, a < b, a >= b, a <= b)
```

| 연산자 | 의미    | 설명          |
|-----|-------|-------------|
| ==  | 같다    | 두 값이 동일하면 참 |
| !=  | 같지 않다 | 두 값이 다르면 참  |

| 연산자 | 의미     | 설명            |
|-----|--------|---------------|
| >   | 크다     | 왼쪽이 크면 참      |
| <   | 작다     | 왼쪽이 작으면 참     |
| >=  | 크거나 작다 | 왼쪽이 크거나 같으면 참 |
| <=  | 작거나 같다 | 왼쪽이 작거나 같으면 참 |

## 제어문 (Control Statements)

단순히 자료구조와 기본 연산자로만 복잡한 프로그램을 구현할 수 없다. 개발자들은 여러 연산의 **반복**, **조건 처리**, **예외처리** 등을 사용하여 복잡한 프로그램을 구현한다.

제어문은 변수 선언, 연산 같은 일반 실행문과 달리 코드의 실행 순서를 바꿀 수 있다. 파이썬에서 제어문은 Block으로 구성되고 **:**와 **Tab** 키를 사용한 들여쓰기로 하나의 **Block**을 묶을 수 있다.

PYTHON

```

if ... :
    Block 1의 코드 1
    Block 1의 코드 2
else ... :
    Block 2의 코드 1
    Blcok 2의 코드 2
    if ...:
        Block 2에 포함된 또 다른 BLock 2-1의 코드 1
        Block 2에 포함된 또 다른 Block 2-1의 코드 2

```

## 조건문

조건문은 **주어진 조건에 따라 실행할 코드를 선택**하는 데 사용된다. 파이썬에서는 **if, elif, else** 문을 사용하여 조건문을 구성한다.

### if 기본 구조

if만을 사용하는 조건문의 기본 구조는 조건의 참(True), 거짓(False)를 판단하여 아래의 코드블록을 실행할지 결정한다. 따라서, 조건으로 구성되는 변수, 연산자 또는함수의 출력은 Bool 자료형으로 변환되어 조건을 판단한다.



```
x = 10

if x > 5:
    print("는 5보다 크다")

if 3:
    print("3은 Bool 자료형으로 변환되었을 때 True입니다.")
```

### ==if ~else 구조==

조건이 참일 때와 거짓일 때 나누어 실행할 때 사용한다. 코드는 반드시 if블록과 else 블록 중 하나를 실행한다. 즉, 두 블록은 조건은 여집합으로 생각할 수 있다.

```
x = 3

if x >= 5:
    print("x는 5보다 큼니다.")
else:
    print("5 미만:")
```

### if~elif~else 구조

여러 조건을 순차적으로 검사하여 판단할 때 사용한다. 판단 순서는 가장 윗 조건부터 판단 하기 때문에 조건 순서를 잘 정해야 한다.

```
score = int(input("점수를 입력하세요: "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print("학점:", grade)
```

## 기본 for문

### ■ 반복문의 필요성

\*\*\*예)

안녕하세요? 만나서 반가워요  
안녕하세요? 만나서 반가워요  
안녕하세요? 만나서 반가워요

```
print("안녕하세요? 만나서 반가워요")  
print("안녕하세요? 만나서 반가워요")  
print("안녕하세요? 만나서 반가워요")
```

- 코드의 수 많은 반복되는 내용을 적는 것은 비효율적이기 때 문에 for문이 필요하다.

## if 문제

### 1. 장학금 판정기

#### 문제

학생의 평균 평점과 이수 학점을 입력받아 장학금 대상 여부를 판정하시오.

장학금 기준은 다음과 같다.

| 조건                        | 결과     |
|---------------------------|--------|
| 평점 4.0 이상이고 이수 학점 15학점 이상 | 전액 장학금 |
| 평점 3.5 이상이고 이수 학점 12학점 이상 | 반액 장학금 |
| 평점 3.0 이상이고 이수 학점 12학점 이상 | 근로 장학금 |
| 그 외                       | 장학금 없음 |

#### 입력 예시 1

4.2 18

#### 출력 예시 1

전액 장학금

#### 입력 예시 2

3.3 15

## 출력 예시 2

근로 장학금

### for문 형식

---

- range(3)은 range(0,3,1)은 같음
- 주의 : for문 끝에는 꼭 (:)을 적어야 한다.
- 파이썬은 반복문의 범위를 (드래그)를 기준으로 한다. 점을 인지하고 있어야한다.

```
for 변수 in range(시작값, 끝값+1, 증가값):  
    이 부분을 반복
```

SHELL

- 예) range() 함수 사용과 내부적 변경

```
for in range(0,3,1):  
    print("만나서 반가워요")
```

PYTHON

```
for i in [0, 1, 2]:  
    print("만나서 반가워요")
```

- i 값 코드 내부 사용

```
for i in range(0,3,1):  
    print("%d: 안녕하세요", %i)
```

PYTHON

### 출력 결과

0 : 안녕하세요  
1 : 안녕하세요  
2 : 안녕하세요

## 다양한 for문의 사용

PYTHON

```
for i in range(10):  
    print(i)  
>>> 0 2 3 4 5 6 7 8 9
```

- 1부터 10까지의 합

PYTHON

```
total = 0  
  
for i in range(1, 11):  
    total += i  
    print(total)  
  
>>> 55
```

- 문자열 한 글자씩 출력

PYTHON

```
word = "Python"  
for ch in word:  
    print(ch)  
>>> P  
>>> y  
>>> t  
>>> h  
>>> o  
>>> n
```

## 실습

1. 실습 1: 아래와 같은 코드를 for문으로 재구성해보자

PYTHON

```
hap = 1+2+3+4+5+6+7+8+9  
print("1에서 10까지의 합계: %d" %hap)
```

2. 500과 1000사이에 있는 홀수의 합계
3. 구구단 2단 코드 작성
4. 1부터 100까지의 5의 배수 출력

## ■ 중첩 for 문

하나의 for문안에 또 하나의 for문을 사용한다.

- 예를 들어 위의 그림처럼 바깥쪽이 3번 도는 동안 안쪽은 각각 2번씩 돌아 총  $3 \times 2 = 6$ 번 반복한다.  
\*\*\*중첩 for문의 기본형식

PYTHON

```
for i in range(0, 3, 1):  
    for k in range(0, 2, 1):  
        print("i값 : %d, k값 : %d" % (i, k))
```

```
i값 : 0, k값 : 0  
i값 : 0, k값 : 1  
i값 : 1, k값 : 0  
i값 : 1, k값 : 1  
i값 : 2, k값 : 0  
i값 : 2, k값 : 1
```

- 처리 순서

외부 변수인 i는 0,1,2로 변경된 후 끝나지만, 내부 변수인 k는 0과 1을 계속 반복

## 중첩 for문 예제

PYTHON

```
for i in range(2, 10, 1):  
    for k in range(1, 10, 1):  
        print("%d x %d = %2d" % (i, k, i*k))  
    print("")
```

출력결과

```
2 x 1 = 2  
2 x 2 = 4  
2 x 3 = 6  
---중략---  
9 x 9 =81
```

# while 문

## while 문의 기본 구조

---

PYTHON

```
while 조건문:
    수행할_문장1
    수행할_문장2
    수행할_문장3
    ...
```

- while 문은 조건문이 참일 동안 아래 문장들을 반복해서 수행
- 예)

SHELL

```
age = 0
while age < 20:
    age = age + 1
    print("제 나이는 %d입니다." %age)
```

```
제 나이는 1살입니다.
제 나이는 2살입니다.
제 나이는 3살입니다.
제 나이는 4살입니다.
...생략...
제 나이는 20살입니다.
```

- 예시를 통해서 알 수 있듯이 age가 20보다 작은 동안 age에 1을 더하면서 구문을 반복한다.

## 무한 루프

---

무한 루프란 무한히 코드를 반복한다는 의미다. 파이썬에서는 while문을 사용해 무한 루프를 구현할 수 있다.

- 무한 루프 적용: while 조건식 : 에 들어가는 조건식을 True로 지정

PYTHON

```
while True:
    수행할_문장1
    수행할_문장2
    ...
```

- 무한 루프의 예

PYTHON

```
>>> while True:
...     print("Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.")
```

출력결과

```
Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.
Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.
Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.
```

- 위 문장은 영원히 출력한다. [ctrl+c]를 눌러 while문을 빠져나갈 수 있다.

## while-else문

---

파이썬의 while문은 else절을 사용 가능하다. while문이 정상적으로 종료 되었을 때 (break로 빠져나가지 않았을 때) else절이 실행된다

PYTHON

```
num = 0
while num < 2:
    print(num)
    num += 1
else:
    print("while 문이 정상 종료되었습니다.")
```

출력

```
0
1
while 문이 정상 종료되었습니다.
```

## for 문과 while 문 비교

---

for 문은 반복할 횟수를 range() 함수에서 결정 후 그 횟수만큼 반복, while 문은 반복 횟수를 결정하기 보다는 조건식이 참일 때 반복하는 방식

- for 문과 비슷하게 사용할 수 있는 while문의 형식

```

변수 = 시작값
while 변수 < 끝값 :
    이 부분을 반복
    변수 = 변수 + 증가값

```

- for문으로 문장을 3회 출력하는 코드

```

for i in range(0, 3, 1) :
    print("%d: 안녕하세요?" %i)

```

- while 문으로 문장을 3회 출력하는 코드

```

i = 0
while i < 3:
    print("%d : 안녕하세요?", %i)
    i += 1

```

## break 문과 continue 문

### break문

while문은 조건문이 참인 동안 반복해서 수행한다. 하지만 강제로 while문을 빠져나가야 하는 경우가 자주 발생한다. 이런 경우 break문을 사용한다.

- break: 조건과 상관 없이 반복문을 바로 빠져나감. 반복이 끝나지 않아도 강제로 종료

```

for i in range(1, 11):
    if i == 5:
        break
    print(i)

```

실행결과

```

1
2
3
4
5

```



- `i == 5`가 되는 순간 `break` 실행해 반복문을 종료한다.
- `break`문은 `for`문에서 동일하게 사용 가능하다.

## continue 문

파이썬 프로그램이 실행되다가 `continue`를 만나면 그 아래의 코드를 수행하지 않고 `while` 문의 조건을 판단하는 곳으로 점프하게 된다.

```
hap, i = 0, 0

for i in range(1, 101) :
    if i % 3 == 0:
        continue

    hap += i
print("1~100의 합계(3의 배수 제외) :", hap)
```

PYTHON

출력 결과

1~100의 합계(3의 배수 제외) : 3367

## 실력 테스트

### 1. 전기요금 계산기

#### 문제

한 달 전기 사용량 `kWh`를 입력받아 전기요금을 계산하시오.

요금은 다음과 같이 누진제로 계산한다.

| 구간           | 요금         |
|--------------|------------|
| 0 ~ 200kWh   | 1kWh당 120원 |
| 201 ~ 400kWh | 1kWh당 214원 |
| 401kWh 이상    | 1kWh당 307원 |

단, 각 구간별로 나누어 계산한다.

예를 들어 `450kWh`를 사용했다면,

$$200 * 120 + 200 * 214 + 50 * 307$$

을 계산한다.

기본요금은 1600원이다.

## 입력

450

## 출력

83750원

## 조건

- 사용량은 0 이상의 정수이다.
- 총 요금은 정수로 출력한다.

## 2. 학점 등급 판정기

### 문제

중간고사, 기말고사, 과제 점수를 입력받아 최종 점수와 학점을 출력하시오.

최종 점수는 다음과 같이 계산한다.

$$\text{최종 점수} = \text{중간고사} * 0.35 + \text{기말고사} * 0.45 + \text{과제} * 0.20$$

학점 기준은 다음과 같다.

| 최종 점수 | 학점 |
|-------|----|
| 90 이상 | A  |
| 80 이상 | B  |
| 70 이상 | C  |
| 60 이상 | D  |

|       |    |
|-------|----|
| 최종 점수 | 학점 |
| 60 미만 | F  |

## 입력

85 90 100

## 출력

최종 점수: 90.25학점: A

## 조건

- 세 점수는 모두 0 이상 100 이하의 정수이다.
- 최종 점수는 소수점 둘째 자리까지 출력한다.

## 3. 미니 ATM 계산기

### 문제

현재 잔액과 거래 종류, 거래 금액을 입력받아 거래 후 잔액을 출력하시오.

거래 종류는 다음과 같다.

| 입력       | 의미 |
|----------|----|
| deposit  | 입금 |
| withdraw | 출금 |
| transfer | 이체 |

처리 규칙은 다음과 같다.

- deposit** 이면 거래 금액만큼 잔액이 증가한다.
- withdraw** 이면 거래 금액만큼 잔액이 감소한다.
- transfer** 이면 거래 금액과 수수료 **500원** 이 함께 빠져나간다.
- 출금 또는 이체 시 잔액이 부족하면 거래를 취소한다.

## 입력 예시 1

```
10000withdraw3000
```

## 출력 예시 1

```
거래 성공 잔액: 7000원
```

## 입력 예시 2

```
10000transfer9800
```

## 출력 예시 2

```
거래 실패 잔액: 10000원
```

## 조건

- 현재 잔액과 거래 금액은 0 이상의 정수이다.
- 거래 종류는 `deposit`, `withdraw`, `transfer` 중 하나이다.

## 리스트

### 리스트의 개념 및 생성 방법

---

파이썬에서 리스트(list)는 여러개 의 값을 하나로 묶어서 저장하는 자료형

- 생성 방법: 리스트를 만들 때는 대괄호([])로 감싸주고 각 요솟값은 쉼표(,)로 구분해 준다.

```
리스트명 = [요소1, 요소2, 요소3, ...]
```

SHELL

- 리스트의 생김새

```
a = []
b = [1, 2, 3]
c = ['life', 'is', 'very', 'beautiful']
d = [1, 2, 'life', 'is']
e = [1, 2, ['life', 'is']]
```

- 리스트에는 어떠한 자료형도 포함 할 수 있다.
- 리스트 저장방법

SHELL

```
a = [0, 0, 0]
a[0] = 3
a[1] = 4
a[2] = 5
print(a)
```

- 리스트 사용 방법

| 각 변수 사용                    | 리스트 사용                |
|----------------------------|-----------------------|
| a, b, c, d = 10, 20 30, 40 | aa = [10, 20 ,30 ,40] |
| a 사용                       | aa[0] 사용              |
| b 사용                       | aa[1] 사용              |
| c 사용                       | aa[2] 사용              |
| d 사용                       | aa[3] 사용              |

- 리스트는 각 인덱스의 시작이 0이다.

## 빈 리스트의 생성과 항목 추가

- append 매서드를 사용한 항목 추가

SHELL

```
aa = []
aa.append(0)
aa.append(0)
aa.append(0)
print(aa)
```

출력결과  
[0,0,0,0]

- for 문을 활용한 항목 추가

```
aa = []  
for i in range(0, 100):  
    aa.append(0)  
len(aa)
```

SHELL

출력 결과  
100  
> len()은 리스트의 항목의 개수를 세는 함수

## 리스트 값에 접근하는 다양한 방법

---

### 1. 음수값으로 접근

```
aa = [10, 20, 30, 40]  
  
print("aa[-1]은 %d, aa[-2]는 %d" % (aa[-1], aa[-2]))
```

SHELL

출력  
aa[-1]은 40, aa[-2]는 30

- 음수값으로 접근하면 뒤 항목부터 접근한다. aa는 aa[-4]까지 접근되므로 aa[-5]는 오류를 발생한다.
- ### 2. 리스트에 접근할 때 콜론(:)을 사용해 범위를 지정

```
aa = [10, 20, 30, 40]  
aa[0:3]  
aa[2:4]
```

SHELL

출력 결과

[10, 20, 30]

[30, 40]

### 3. 콜론의 앞이나 숫자 생략

SHELL

```
aa = [10, 20, 30, 40]
```

```
aa[2:]
```

```
aa[:2]
```

출력 결과

[30, 40]

[10, 20]

### 4. 리스트끼리 덧셈, 곱셈 연산

SHELL

```
aa = [10, 20, 30]
```

```
bb = [40, 50, 60]
```

```
aa + bb
```

```
aa * 3
```

출력 결과

[10, 20, 30, 40, 50, 60]

[10, 20, 30, 10, 20, 30, 10, 20, 30]

### 5. 리스트의 항목 건너뛰며 추출

SHELL

```
aa = [1, 2, 3, 4, 5, 6, 7]
```

```
aa = [::2]
```

```
aa = [::-2]
```

```
aa = [::-1]
```

출력 결과

[1, 3, 5, 7]

[7, 5, 3, 2]

[7, 6, 5, 4, 3, 2, 1]

## 리스트 값의 변경

---

### 1. 두 번째에 위치한 값을 1개 변경하는 방법

```
aa = [1, 2, 3]
a[1] = 200
a
```

출력 결과  
[1, 200, 3]

### 2. 두 번째 값인 2를 200과 300이라는 2개의 값으로 변경

```
a = [1, 2, 3]
a[1:2] = [200, 300]
a
```

SHELL

출력 결과  
[1, 200, 300, 3]

### 3. 그냥 a[1] 사용시

```
a = [1, 2, 3]
a[1] = [200, 300]
a
```

출력 결과  
[1, [200, 300], 3]

### 4. 리스트 값 삭제

```
a = [1, 2, 3, 4, 5]; del(a[1]); a
a = [1, 2, 3, 4, 5]; a[1:3] = []; a
```

SHELL



출력 결과

[1, 3, 4, 5]

[1, 5]

## 5. 리스트 자체 삭제

SHELL

```
a = [1, 2, 3]; a = []; a
a = [1, 2, 3]; a = None; a
a = [1, 2, 3]; del(a); a
```

출력 결과

[]

아무것도 안나옴

오류 발생

## 리스트 조작 함수

| 함수        | 설명                                          | 사용법                 |
|-----------|---------------------------------------------|---------------------|
| append()  | 리스트 맨 뒤에 항목을 추가한다.                          | 리스트명.append(값)      |
| pop()     | 리스트 맨 뒤의 항목을 빼낸다. (리스트에서 해당 항목이 삭제된다)       | 리스트명.pop()          |
| sort()    | 리스트의 항목을 정렬한다.                              | 리스트명.sort()         |
| reverse() | 리스트 항목의 순서를 역순으로 만든다.                       | 리스트명.reverse()      |
| index()   | 지정한 값을 찾아 해당 위치를 반환한다.                      | 리스트명.index(찾을값)     |
| insert()  | 지정된 위치에 값을 삽입한다.                            | 리스트명.insert(위치, 값)  |
| remove()  | 리스트에서 지정한 값을 삭제한다. 단, 값이 여러 개면 첫 번째 값만 지운다. | 리스트명.remove(지울값)    |
| extend()  | 리스트 뒤에 리스트를 추가한다. 리스트의 더하기(+) 연산과 기능이 동일하다. | 리스트명.extend(추가할리스트) |
| count()   | 리스트에서 해당 값의 개수를 센다.                         | 리스트명.count(찾을값)     |
| clear()   | 리스트의 내용을 모두 지운다.                            | 리스트명.clear()        |
| del()     | 리스트에서 해당 위치의 항목을 삭제한다.                      | del(리스트명[위치])       |
| len()     | 리스트에 포함된 전체 항목의 개수를 센다.                     | len(리스트명)           |
| copy()    | 리스트의 내용을 새로운 리스트에 복사한다.                     | 새리스트 = 리스트명.copy()  |
| sorted()  | 리스트의 항목을 정렬해서 새로운 리스트에 대입한다.                | 새리스트 = sorted(리스트)  |

## 2차원 리스트

---

1차원 리스트를 여러 개 연결한 것, 첨자를 2개 사용한다.

### 1. 예시

```
#1차원
a = [1,2,3]
#2차원 리스트
a = [[1,2,3,4],
      [5,6,7,8],
      [9,10,11,12]]
```

SHELL

## 튜플

- 순서대로 묶어서 저장하는 자료형
- 특징
  1. 변경 불가능: 한 번 만들면 값을 수정, 추가, 삭제할 수 없음-> 읽기만 가능해 읽기 전용 자료를 저장할 때 사용
  2. 순서가 있음: 인덱스로 접근 가능
  3. 서로 다른 자료형 저장 가능

## 튜플의 생성

---

### 1. 튜플은 소괄호 ()로 생성

- 튜플은 소괄호() 생략 가능, 항목이 하나인 튜플은 뒤에 쉼표(,)붙임

```
t1 = (10 , 20 ,30); t1
t2 = 10, 20, 30; t2
t3 = (10,); t3
t4 = 10,; t4
```

SHELL

출력

```
(10, 20, 30)
(10, 20, 30)
(10,)
(10,)
```

## 튜플의 사용

---

기본적으로 튜플 값을 바꿀수 없다는 것을 제외하면 리스트 사용법과 거의 같다.

### 1. 튜플 범위에 접근

```
t1 = (10 , 20 ,30, 40)
t1[1:3]
t1[1:]
t[3]
```

PYTHON

출력

```
(20, 30)
(20, 30, 40)
(10,20, 30)
```

### 2. 튜플의 덧셈 및 곱셈 연산

```
t1 = (10 , 20 )
t2 = ('A', 'B')
t1 + t2
t1*3
```

PYTHON

출력

```
(10, 20, 'A', 'B')
(10,20,10,20,10,20)
```

## 튜플-> 리스트-> 튜플 변환

---

```
my_tuple = (10, 20, 30)
my_list = list(my_tuple) #튜플을 리스트로 변환한 값을 저장
my_list.append(40)
my_tuple = tuple(my_list)
my_tuple
```

PYTHON

출력

(10, 20, 30, 40)

## 딕셔너리(dictionary)

- 키(key) : 값(value) 쌍으로 데이터를 저장하는 자료형
- 이름표(키)로 값을 찾는 구조

### 딕셔너리의 개념

#### 1. 쌍 2개가 하나로 묶인 자료구조

- 중괄호 {}로 묶어 구성, 키(key)와 값(value)의 쌍으로 구성

딕셔너리변수 = {키: 값1, 키2:값2, 키3: 값3, ...}

SHELL

### 딕셔너리의 생성

- 키와 값은 사용자가 지정하는 것이지 규정은 없음
- 주의점: 딕셔너리에는 순서가 없어 생성한 순서대로 딕셔너리가 구성되어 있다는 보장 없음

```
dic = {1:'a', 2:'b', 3:'c', 4:'d'}  
dic
```

SHELL

출력 결과

{1:'a', 2:'b', 3:'c', 4:'d'}

#### 2. 딕셔너리 새로운 데이터 추가

```
dic = {1:'a', 2:'b', 3:'c', 4:'d'}  
dic[5] = 'e'  
print(dic)
```

SHELL

출력 결과

{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}

### 3. 내부 데이터 삭제

SHELL

```
dic = {1:'a', 2:'b', 3:'c', 4:'d'}  
del(dic[2])  
print(dic)
```

출력 결과

```
{1: 'a', 3: 'c', 4: 'd'}
```

4. 동일한 키를 가진 인덱스를 저장할 때 동일한 키를 갖는 딕셔너리를 생성하는 것이 아니라 마지막에 있는 키가 적용

```
dic = {1:'a', 2:'b', 3:'c', 4:'d', 1 : 'f'}  
print(dic)
```

```
{1: 'f', 2: 'b', 3: 'c', 4: 'd'}
```

## 딕셔너리 접근

---

### 1. 키로 값에 접근

SHELL

```
student= {'학번' : 100, '이름' : '강호동', '학과' : '전기전자'}  
  
print(student['학번'])  
print(student['이름'])  
print(student['학과'])
```

출력 결과

100

강호동

전기전자

### 2. 딕셔너리명.get(키) 함수를 사용해 키로 값에 접근

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}

print(student.get('이름'))
```

출력 결과  
강호동

- `딕셔너리명[키]` 와 `딕셔너리명.get(키)`는 결과 같음
- `딕셔너리명[키]` 는 없는 키를 호출할 경우 오류가 발생하지만 `딕셔너리명.get(키)`는 없는 키를 호출하면 아무것도 반환하지 않음

3. `딕셔너리명.keys()` 함수를 사용해 키로 값에 접근 : 딕셔너리의 키 값들 출력

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}

print(student.keys())
```

출력 결과  
`dict_keys(['학번', '이름', '학과'])`

4. `딕셔너리명.items()` 함수를 사용하면 튜플 형태로도 구할 수 있음

```
student= {'학번' : 100, '이름': '강호동', '학과': '전기전자'}
print(student.items())

dict_items([('학번', 100), ('이름', '강호동'), ('학과', '전기전자')])
```